

# Ejercicios de Apache Kafka

## Big Data Processing II

Máster Universitario en Análisis de Datos Deportivos

Curso 2026-2027

Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández



**CC BY-SA 4.0**

Atribución-CompartirIgual

Depósito institucional BURJC Digital

<https://burjcdigital.urjc.es>

## Información de licencia y reutilización

Material docente en abierto de la Universidad Rey Juan Carlos.

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández.  
Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons  
Atribución-CompartirIgual 4.0 Internacional, disponible en  
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

La licencia se aplica al contenido original de la obra. Los logotipos, marcas, citas y materiales de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

## TEMA 2 – Apache Kafka

Se proponen diferentes ejercicios de ingesta y procesamiento de datos en streaming aplicados a un escenario de analítica deportiva. Para ello se utilizará **Apache Kafka** como sistema de mensajería distribuida. Se desarrollará un producer que publique mediciones de telemetría deportiva en un topic y distintos consumers que las consuman para **monitorizar condiciones de rendimiento y fisiología en tiempo real**.

En particular, se trabajará con un conjunto de datos de telemetría recogidos a través de sensores durante **3 partidos de fútbol para 15 jugadores**. Las mediciones incluyen parámetros de posición, velocidad, aceleración y frecuencia cardíaca. La generación en tiempo real de las mediciones se simulará a partir del script de Python *producer.py* y el archivo CSV *sensores\_deportivos\_fatiga.csv* proporcionados.

### Ejercicio 1. Publicación de eventos con Kafka

Se pide implementar un **producer** que publique eventos con estas mediciones de telemetría en un topic de Kafka.

#### Apartado 1.1. Diseño del topic y del esquema del mensaje

Define el nombre del topic donde se publicarán los datos en streaming. Se recomienda seguir una convención de nombres jerárquica y con versionado de esquema.

A continuación, diseña el esquema de mensajes que se enviarán al topic. Debe contener los siguientes elementos:

- **Key:** debe elegirse una clave que permita preservar el orden de los eventos dentro de las particiones de un topic.
- **Value:** debe incluir todos los campos de medición y otros metadatos del mensaje relevantes. Se recomienda utilizar JSON e incorporar la versión del esquema de mensajes y un identificador único del evento.
- **Timestamp:** debe utilizarse el tiempo de evento asociado a cada medición y no el instante en el que se envía el mensaje.

#### Apartado 1.2. Implementación del producer

Modifica el script *producer.py* para que publique eventos en el topic de Kafka. Para ello, implementa las modificaciones necesarias en las líneas indicadas en el archivo. El identificador del evento puede generarse a partir de un [uuid](#).

## Ejercicio 2. Consumo de eventos con Kafka

Se pide implementar distintos consumers de Kafka que se suscriban al topic definido en el ejercicio anterior y procesen en tiempo real los eventos publicados.

### Apartado 2.1. Lectura de eventos

Implementa un consumer en Python que se suscriba al topic definido y lea los mensajes de forma continua. Para ello, se debe deserializar el contenido JSON de los mensajes enviados a los topics y mostrar por pantalla cada evento recibido.

### Apartado 2.2. Alertas de frecuencia cardiaca

Implementa un segundo consumer que se suscriba al mismo topic e imprima únicamente por pantalla aquellos eventos que superen el umbral de frecuencia cardiaca de 180 bpm con una velocidad menor a 10 km/h.

## Entregables

Se deberá entregar un único archivo comprimido en formato .zip en el Aula Virtual con los siguientes archivos Python:

- **producer.py**: Script del producer modificado para publicar eventos en el topic definido.
- **consumer\_print.py**: Consumer que se suscribe al topic y muestra por pantalla cada evento recibido.
- **consumer\_alert.py**: Consumer que se suscribe al topic e imprime únicamente los eventos que superen o igualen el umbral de frecuencia cardiaca configurado.

Debe incluirse una **memoria explicativa** en la que se describa el diseño del topic y el esquema de los mensajes empleados. Se deben especificar los requisitos del entorno y una guía clara de instalación y ejecución, indicando cómo lanzar el producer y los consumers.

## MATERIAL DOCENTE EN ABIERTO